

PLANO DE PROJETO DE SOFTWARE

Cliente: Renato Simões Moreira

Responsáveis no cliente: Renato Simões Moreira

Responsáveis pelo projeto no IFSC: Marcela Bianchini Beduschi Caitano

Projeto: Reconhecimento de LIBRAS em Tempo Real

Versão: 1.0

Unidade Curricular: Atividades de Extensão II

Professor: SAULO VARGAS

GASPAR

2026

SUMÁRIO

1 INTRODUÇÃO	3
1.1 Objetivo geral	3
1.2 Objetivos específicos	3
2 LEVANTAMENTO DOS REQUISITOS	4
2.1 Visão geral do negócio	4
2.2 Requisitos de usuário	4
2.3 Requisitos de sistemas	4
2.3.1 Requisitos Funcionais	5
2.3.2 Requisitos Não-Funcionais	5
2.3.3 Requisitos de Negócio	5
3 MODELAGEM DO SISTEMA	6
3.1 Diagramas de casos de uso	6
3.1.1 Especificação dos casos de uso	6
3.2 Diagramas de classe	7
3.3 Diagramas Entidade Relacionamento (ER)	7
4 MAPA DE RASTREABILIDADE ENTRE REQUISITOS E CASOS DE USO	7
5 PROTOTIPAÇÃO	7
6 TECNOLOGIAS E CRONOGRAMA	8
6.1 Tecnologias	8
6.2 Cronograma	8
6.3 Implementação técnica	8
6.4 Limitações das tecnologias e do desenho atual	8
6.5 Erros e obstáculos	8
REFERÊNCIAS	8

1 INTRODUÇÃO

O aplicativo de “Reconhecimento de Libras em Tempo Real” pode ser uma ferramenta valiosa para uma ampla variedade de empresas e instituições que desejam promover a inclusão e a comunicação eficaz entre pessoas surdas e ouvintes, como: escolas e universidades, empresas e locais de trabalho, hospitais e clínicas, eventos e conferências, hotéis, entre outros.

Um dos desafios enfrentados por pessoas surdas e com deficiência auditiva é a dificuldade de comunicação eficaz com os ouvintes. A linguagem de sinais, como a Libras, é uma forma rica e expressiva de comunicação, mas a falta de compreensão e conhecimento da linguagem frequentemente resulta em isolamento, exclusão e desigualdade. A comunicação fluida e natural entre pessoas surdas e ouvintes é essencial para a plena inclusão em diversas situações, desde o cotidiano até o educacional e o profissional.

Nesta primeira etapa, o sistema tem como escopo o reconhecimento de um subconjunto de gestos em Libras, incluindo letras do alfabeto manual (datilologia) e expressões léxico-fixas, permitindo que gestos correspondentes às classes definidas no modelo treinado (25 rótulos em labels.txt: A, Ajuda, B, C, Casa, D, E, Eu te amo, F, G, I, K, L, M, N, O, Oi, P, Q, R, S, T, U, V, W) sejam exibidos como texto na tela em tempo real. A implementação dessa tecnologia teria um impacto transformador na vida das pessoas surdas e com deficiência auditiva. Ao utilizar o aplicativo, as seguintes vantagens seriam alcançadas:

- a) Comunicação Efetiva:** O aplicativo permitirá uma comunicação imediata e eficaz entre pessoas surdas e ouvintes, superando as barreiras linguísticas. Isso promoverá um fluxo mais natural, permitindo que as pessoas surdas expressassem ideias, pensamentos e sentimentos de maneira efetiva e compreendida.
- b) Inclusão Social:** A tecnologia de reconhecimento de gestos em Libras em tempo real facilitará a participação em uma variedade de contextos sociais, como reuniões, eventos, conversas cotidianas e até mesmo nas redes sociais. Isso ajuda a combater o isolamento social,

permitindo que as pessoas surdas se envolvam ativamente na sociedade.

- c) **Acesso à Educação e Informação:** O aplicativo abre portas para a comunicação e o aprendizado ao permitir que letras e outros rótulos cobertos pelo modelo sejam reconhecidos a partir do vídeo e exibidos como texto, facilitando a interação em sala de aula, em tutoriais e em situações em que o alfabeto manual é utilizado.
- d) **Empoderamento Profissional:** A tecnologia facilitará a comunicação no dia a dia profissional ao permitir a soletração de nomes, siglas e termos em Libras com tradução imediata para texto, podendo levar a oportunidades de emprego mais amplas e progressão na carreira.
- e) **Promoção da Conscientização e Educação:** A adoção do aplicativo incentiva a conscientização sobre a importância da inclusão e da comunicação com pessoas surdas. Isso criará um ambiente mais abrangente e inclusivo, em que a linguagem de sinais é valorizada e respeitada.

1.1 Objetivo geral

O objetivo geral do projeto “Reconhecimento de Libras em Tempo Real” é desenvolver um aplicativo inovador que utiliza inteligência artificial para facilitar a comunicação imediata e eficaz entre pessoas surdas e ouvintes, a partir do reconhecimento em tempo real de gestos em Libras correspondentes às classes treinadas no modelo (datilologia e expressões fixas incluídas no treino), superando barreiras linguísticas e promovendo a inclusão social.

1.2 Objetivos específicos

- a) **Desenvolvimento da Plataforma de Reconhecimento de Gestos:** Criar um sistema robusto de reconhecimento de gestos baseado em visão computacional e IA, capaz de capturar e interpretar com precisão os gestos cobertos pelo modelo (labels.txt, 25 rótulos), incluindo letras do alfabeto manual e expressões fixas treinadas, garantindo uma tradução confiável e em tempo real.

- b) Treinamento do Modelo de Inteligência Artificial:** Coletar, organizar e, quando aplicável, ampliar o conjunto de imagens de gestos alinhadas às classes do modelo (25 rótulos na versão atual), apoiando retreinos e avaliação contínua da precisão na tradução de cada gesto em texto.
- c) Integração de Recursos Multissensoriais:** Incorporar recursos visuais na interface do aplicativo, exibindo em tempo real o texto correspondente a cada gesto reconhecido, com perspectiva futura de recursos auditivos (síntese de voz).
- d) Desenvolvimento de Interface Intuitiva:** Criar uma interface de usuário intuitiva com acesso à câmera, troca entre câmera frontal e traseira, menu de ajuda (FAQ) e visualização contínua da tradução dos gestos reconhecidos, adaptável a smartphones e tablets.
- e) Tradução dos gestos do modelo em tempo real:** Implementar a tradução dos gestos das classes em labels.txt para texto em tempo real.
- f) Armazenamento de Histórico:** Permitir que os usuários tenham um histórico das conversas para facilitar a comunicação contínua, em versões posteriores do aplicativo.
- g) Testes e Aperfeiçoamento Contínuo:** Realizar testes rigorosos do aplicativo com uma variedade de usuários, incluindo pessoas surdas, ouvintes e especialistas em linguagem de sinais. Coletar feedback e realizar ajustes para melhorar a precisão da tradução, usabilidade e acessibilidade.
- h) Promoção da Conscientização e Acessibilidade:** Criar campanhas de conscientização para destacar a importância da inclusão e comunicação entre pessoas surdas e ouvintes. Colaborar com organizações e comunidades surdas para garantir que o aplicativo atenda às suas necessidades e expectativas.
- i) Lançamento e Distribuição:** Lançar o aplicativo em plataformas móveis populares, como Android (e, futuramente, iOS), e disponibilizá-lo para download gratuito, permitindo que pessoas de diversos contextos se beneficiem dessa tecnologia inclusiva.

2 LEVANTAMENTO DOS REQUISITOS

Para levantamento dos requisitos do projeto foi realizada uma entrevista com Renato Simões Moreira, cliente e professor vinculado ao IFSC - Instituto Federal de Santa Catarina, a fim de compreender a necessidade e os benefícios do sistema a ser desenvolvido e levantar as tarefas que o sistema precisa executar. Foi realizada uma entrevista, no dia 30/03/2026.

2.1 Visão geral do negócio

O aplicativo mobile “Reconhecimento de Libras em Tempo Real” utiliza visão computacional e inteligência artificial para reconhecer gestos em Libras capturados pela câmera, incluindo datilologia e expressões fixas (conforme as 25 classes em labels.txt) e exibir a tradução em texto na tela em tempo real. O escopo atual abrange: captura de vídeo ao vivo, detecção de mãos, classificação dos gestos por meio de um modelo YOLO em TensorFlow Lite (.tflite) com ficheiro de rótulos (labels.txt), enquanto a detecção da mão e dos landmarks é feita com MediaPipe Hands e exibição do resultado ao usuário. Em fases futuras, estão previstas a ampliação do vocabulário (sinais além do alfabeto), a tradução bidirecional (fala ↔ Libras), o histórico de conversas e a síntese de voz.

2.2 Requisitos de usuário

Código	Nome	Descrição
RU01	Ver gesto reconhecido em tempo real	Como usuário, desejo que o aplicativo reconheça meus gestos correspondentes às classes do modelo pela câmera e mostre na tela o texto do rótulo correspondente em tempo real.
RU02	Ver feedback visual da mão	Como usuário, desejo ver na tela os pontos detectados da minha mão sobre a imagem da câmera para saber se o sistema está acompanhando corretamente.
RU03	Usar o app sem	Como usuário, desejo que o reconhecimento

	internet	de gestos funcione offline, utilizando apenas o dispositivo.
RU04	Alternar câmera	Como usuário, desejo alterar entre a câmera frontal e a traseira para melhor enquadrar a mão e o gesto.
RU05	Consultar ajuda (FAQ)	Como usuário, desejo abrir uma página de ajuda com perguntas frequentes sobre o uso do aplicativo, permissões e privacidade.

2.3 Requisitos de sistemas

Como realizar a Priorização

- Essencial - Requisitos que devem ser trabalhados primeiro para que o projeto seja viabilizado (MVP, Mínimo Produto Viável).
- Importante - Requisitos que são complementares aos que foram priorizados que são correlacionados após a implementação do MVP.
- Desejável - Demais requisitos e funcionalidades a serem implementadas no projeto.

2.3.1 Requisitos Funcionais

Código	Nome	Descrição	Prioridade
RF01	Captura de vídeo em tempo real	O sistema deve capturar o fluxo de vídeo da câmera do dispositivo em tempo real para análise.	Essencial
RF02	Deteção de mão	O sistema deve detectar e rastrear mãos no quadro de vídeo utilizando visão computacional.	Essencial
RF03	Classificação dos	O sistema deve classificar	Essencial

	gestos do modelo	os gestos detectados com base em um modelo de IA, reconhecendo as classes treinadas no modelo TensorFlow Lite (best_float32.tflite e labels.txt), incluindo letras do alfabeto manual e outras expressões fixas, retornando o rótulo correspondente.	
RF04	Exibição da tradução em texto	O sistema deve exibir na interface o texto (rótulo) resultante da tradução do gesto reconhecido de forma contínua e atualizada.	Essencial
RF05	Overlay da mão na tela	O sistema deve desenhar os landmarks da mão sobre a imagem da câmera para feedback visual ao usuário.	Importante
RF06	Configuração de opções de saída	O sistema deve poder configurar se a saída será apenas texto, apenas voz ou ambos.	Desejável
RF07	Síntese de voz	O sistema deve converter o texto traduzido em áudio por meio de síntese de voz quando habilitado.	Desejável

RF08	Histórico de conversas	O sistema deve armazenar e permitir consulta do histórico de traduções por sessão ou por período.	Desejável
RF09	Reconhecimento de fala	O sistema deve capturar a fala do usuário e convertê-la em texto para posterior exibição em Libras.	Desejável
RF10	Exibição de sinal em Libras a partir da fala	O sistema deve exibir representação em Libras a partir do texto obtido pela fala.	Desejável

2.3.2 Requisitos Não-Funcionais

Classificação:

- Desempenho
 - Usabilidade
 - Disponibilidade
- Segurança
 - Confiabilidade
- Manutenibilidade
- Operacional

Código	Nome	Descrição	Classificação	Prioridade
RNF01	Tempo de resposta	A tradução do gesto para texto deve ser exibida em até 2 segundos após a execução do gesto.	Desempenho	Essencial

RNF02	Uso em dispositivos móveis	O aplicativo deve rodar em smartphones e tablets Android.	Operacional	Essencial
RNF03	Execução offline do reconhecimento	O reconhecimento de gestos do modelo deve funcionar sem conexão com a internet.	Disponibilidade e	Essencial
RNF04	Interface intuitiva	A interface deve permitir uso com poucas instruções, com elementos visíveis e acessíveis.	Usabilidade	Importante
RNF05	Estabilidade da câmera	O aplicativo não deve encerrar abruptamente por falhas de permissão ou inicialização da câmera, deve tratar erros e informar ao usuário.	Confiabilidade	Importante
RNF06	Manutenção do modelo	O modelo de gestos deve ser substituído sem alteração obrigatória do código principal.	Manutenibilidade	Importante
RNF07	Privacidade dos dados	Imagens e vídeos devem ser processados	Segurança	Desejável

		localmente, sem envio obrigatório para servidores externos.		
--	--	---	--	--

2.3.3 Requisitos de Negócio

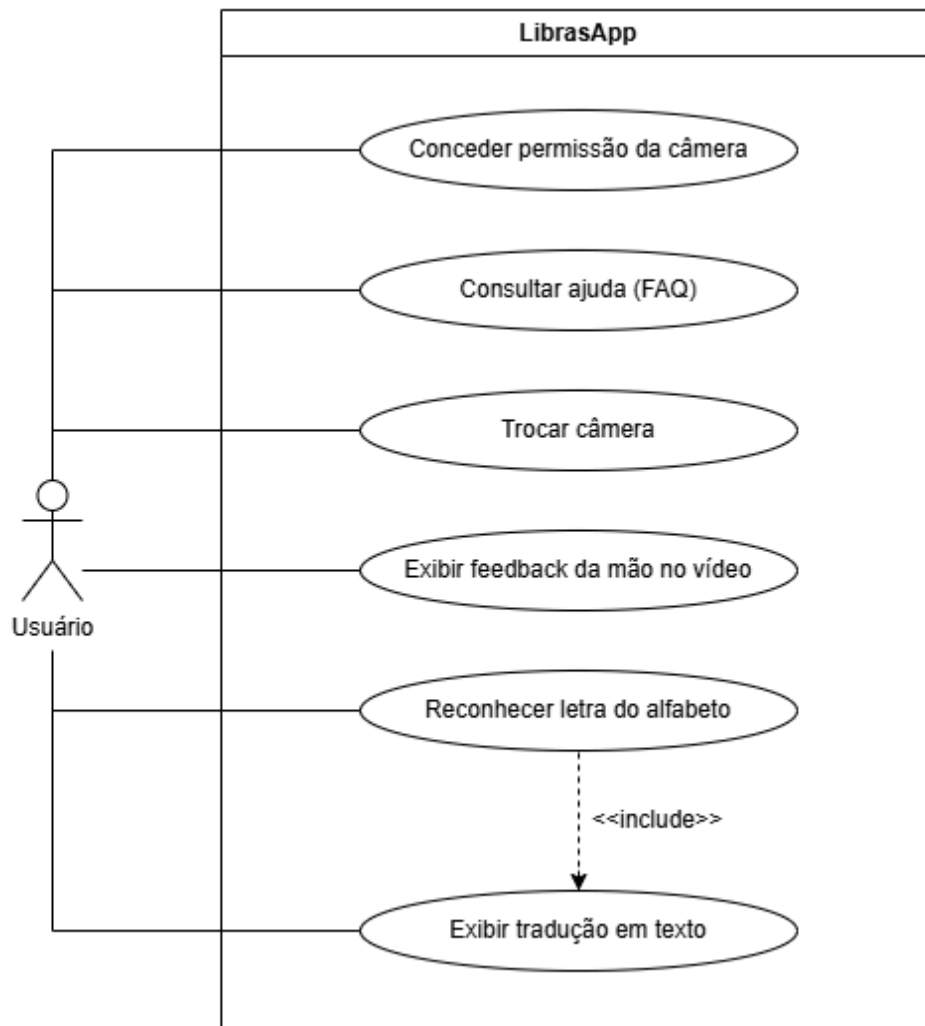
Código	Nome	Descrição
RN01	Inclusão e acessibilidade	O sistema deve contribuir para a inclusão de pessoas surdas e com deficiência auditiva, por meio do reconhecimento de gestos do vocabulário fixo do modelo (datilologia e expressões incluídas no treino), exibido como texto.
RN02	Uso em múltiplos contextos	O sistema deve ser utilizável em ambientes como educação, saúde, trabalho, eventos e serviços.
RN03	Distribuição gratuita	O aplicativo deve ser disponibilizado para download gratuito em lojas de aplicativos.
RN04	Conscientização sobre Libras	O projeto deve promover conscientização sobre a importância da Libras e comunicação acessível.

3 MODELAGEM DO SISTEMA

3.1 Diagramas de casos de uso

O usuário precisa conceder permissão de câmera (UC04) para que o sistema capture vídeo. Com a câmera ativa, o usuário pode trocar entre câmera frontal e traseira (UC05) e abrir a tela de ajuda com FAQ (UC06). A detecção de mãos e o

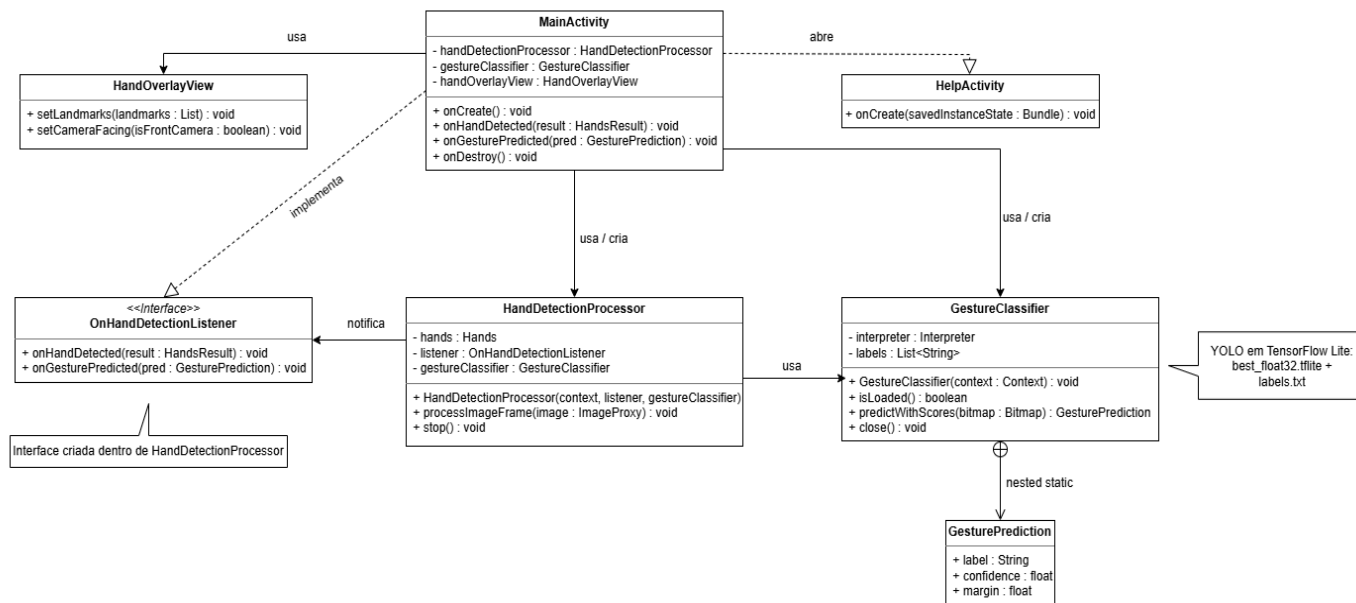
feedback visual de landmarks (UC03) vêm do MediaPipe no mesmo fluxo de imagem usado para enviar quadros ao classificador. O reconhecimento do gesto (UC01) produz o rótulo exibido na interface (UC02). UC02 depende de UC01, UC03 compartilha o fluxo de vídeo com UC01, UC04 habilita o pipeline, UC05 e UC06 são fluxos complementares de uso.



3.2 Diagramas de classe

O diagrama de classes da implementação atual inclui: MainActivity (CameraX, permissões, menu de ajuda, callbacks para UI), HandDetectionProcessor (conversão de ImageProxy em Bitmap, envio ao MediaPipe Hands, coordenação com GestureClassifier na thread de análise), GestureClassifier (carregamento de best_float32.tflite e labels.txt, inferência YOLO), HandOverlayView (desenho dos landmarks), HelpActivity (texto de ajuda). A

comunicação entre processamento e UI ocorre pela interface OnHandDetectionListener (declarada dentro de HandDetectionProcessor), e o resultado da classificação é exposto como GesturePrediction. O modelo e os rótulos residem em assets no APK. Evoluções futuras (reconhecimento dinâmico, interfaces adicionais) podem ser modeladas sem alterar esta base.



3.3 Diagramas Entidade Relacionamento (ER)

Na versão atual do sistema, não há persistência em banco de dados para usuários, sessões ou histórico, modelo e predições residem nos assets do aplicativo. Por isso, não há modelo ER de implementação obrigatório. Opcionalmente, pode-se registrar um ER futuro para histórico de traduções (entidades Usuário, Sessão, Tradução), alinhado a RF08.

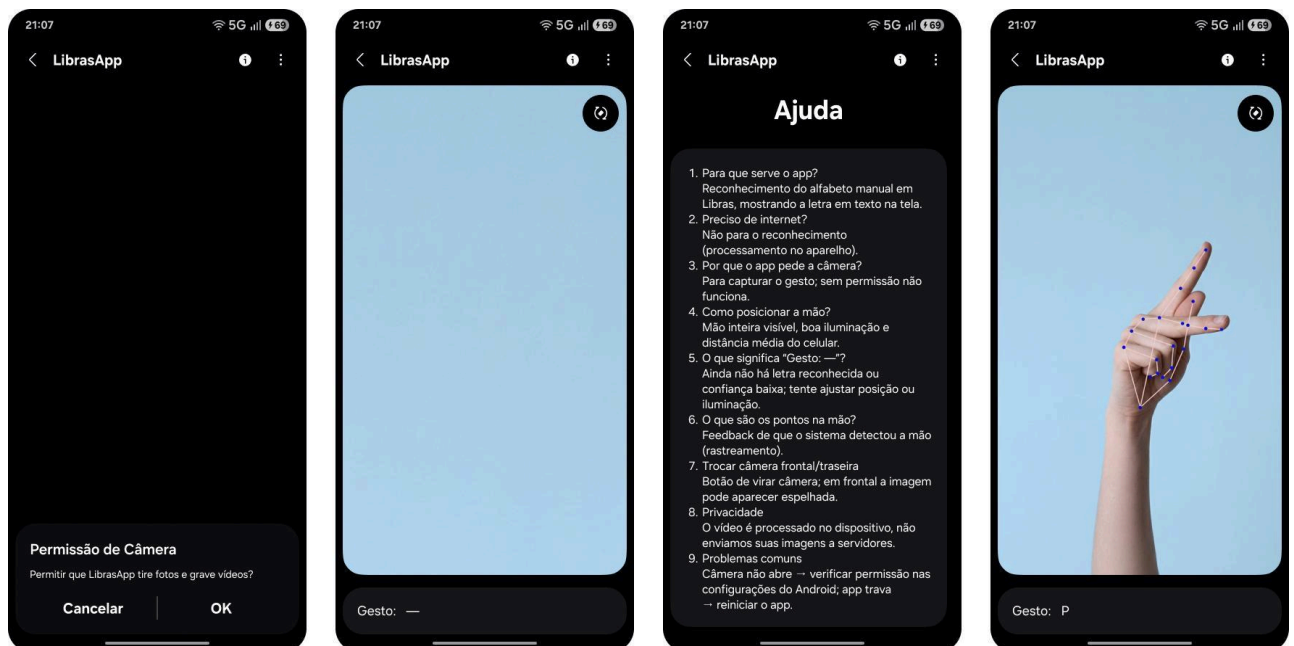
4 MAPA DE RASTREABILIDADE ENTRE REQUISITOS E CASOS DE USO

Requisitos X Casos de Uso	[UC01]	[UC02]	[UC03]	[UC04]	[UC05]	[UC06]
[RF01]	X			X	X	

[RF02]	X		X			
[RF03]	X					
[RF04]		X				
[RF05]			X			

5 PROTOTIPAÇÃO

Esta seção apresenta o protótipo de interface do aplicativo LibrasApp, elaborado no Figma com base no One UI Design Kit, em tema escuro. As telas ilustram o fluxo principal: solicitação de permissão da câmera, visão da câmera antes/durante a detecção (com indicação “Gesto: —” ou rótulo reconhecido), botão para trocar a câmera, tela de Ajuda com perguntas frequentes sobre uso, privacidade e feedback visual, e o estado de reconhecimento com landmarks da mão e exibição do rótulo reconhecido (ex.: Gesto: P). O objetivo é validar disposição dos elementos, acessibilidade da informação e coerência com os requisitos funcionais de captura, detecção, classificação e exibição do resultado.



6 TECNOLOGIAS E CRONOGRAMA

6.1 Tecnologias

Plataforma e linguagem:

- Android (aplicativo nativo)
- Java
- Gradle (build)
- Android Studio (ambiente de desenvolvimento)

Interface e componentes Android

- AndroidX - AppCompatActivity, Material Components, Activity, CoordinatorLayout, LinearLayout
- CameraX - núcleo da câmera, extensão Camera2, integração com ciclo de vida, Camera View (pré-visualização)

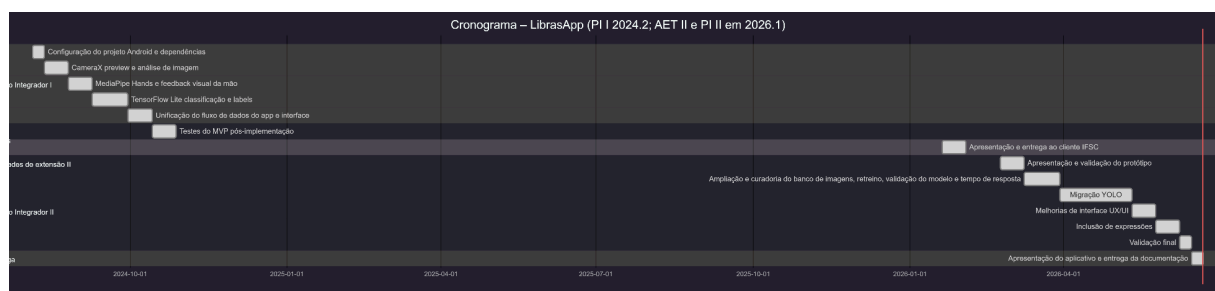
Visão computacional e aprendizado de máquina

- MediaPipe Hands - detecção e rastreamento de mãos (landmarks)
- TensorFlow Lite - inferência do modelo de classificação de gestos (.tflite) no dispositivo
- Roboflow - anotação, organização, versionamento do dataset de gestos em Libras e exportação no formato compatível com o pipeline de treino YOLO
- YOLO - modelo treinado guardado em best_float32.tflite, com rótulos em labels.txt

6.2 Cronograma

O cronograma está organizado em dois períodos de implementação (PI I e PI II), separados por testes do MVP ao final da primeira implementação. Após a PI I e os testes do MVP, registra-se a conclusão do ciclo de Atividades de Extensão II, com apresentação e entrega formal ao cliente, antes do conjunto de trabalhos descritos como PI II. Na PI II concentram-se melhorias e evolução do aplicativo: ampliação e curadoria de dados, retreino e validação do modelo, migração para o fluxo com YOLO, evolução planejada para expressões, ajustes de interface (UX/UI) e, em seguida, validação final, consolidação, documentação e apresentação para

encerramento acadêmico da disciplina. A figura apresenta o diagrama de Gantt e, a tabela detalha cada grupo de tarefas e as respectivas entregas esperadas.



Tarefa	Objetivo	Principais Entregas
Implementação - PI I: Configuração do projeto Android e dependências	Preparar o ambiente e as bibliotecas do app.	Projeto Android estruturado, Gradle, dependências (AndroidX, CameraX, MediaPipe, TensorFlow Lite).
Implementação - PI I: CameraX preview e análise de imagem	Obter fluxo de vídeo em tempo real para processamento.	Pré-visualização da câmera e análise de quadros.
Implementação - PI I: MediaPipe Hands e feedback visual da mão	Detectar e rastrear a mão no vídeo.	Landmarks integrados e feedback visual na tela.
Implementação - PI I: TensorFlow Lite classificação e labels	Reconhecer os gestos do alfabeto manual no dispositivo.	Inferência com .tflite e labels.txt nos assets.
Implementação - PI I: Unificação do fluxo de dados do app e interface	Fechar o pipeline (câmera → detecção → classificação → UI).	Aplicativo coeso, resultado do gesto na interface e fluxo estável.
Testes MVP: Testes pós-implementação	Validar o núcleo funcional após a PI I.	Registro de testes, lista de falhas e correções

		prioritárias.
Entrega: Apresentação e entrega ao cliente	Encerrar o ciclo da extensão.	Apresentação (Atividades de Extensão II) e entrega formal ao cliente (Renato Simões Moreira), conforme combinado com o IFSC.
Melhorias e evolução - Sprint 1 - PI II: Apresentação e validação do protótipo	Demonstrar o estado do app e receber feedback para a evolução.	Apresentação do protótipo, leitura de letras/gestos, feedback e prioridades.
Melhorias e evolução - Sprint 2 - PI II: Ampliação e curadoria do banco de imagens, retreino, validação do modelo e tempo de resposta	Melhorar a base de dados para treino, aumentar acerto e desempenho percebido.	Conjunto de imagens ampliado e organizado, modelo retreinado, ajustes para reduzir atraso entre gesto e texto na tela.
Melhorias e evolução - Sprint 3 - PI II: Migração para YOLO	Substituir a classificação anterior por um fluxo baseado em YOLO, alinhado ao modelo em uso no app.	Integração do modelo YOLO em .tflite, labels.txt e inferência no dispositivo.
Melhorias e evolução - Sprint 4 - PI II: Melhorias de interface UX/UI	Aproximar o app do protótipo e da usabilidade desejada.	Ajustes de telas, fluxos e consistência visual.
Melhorias e evolução - Sprint 5 - PI II: Inclusão de expressões	Ampliar o escopo além do gesto letra a letra.	Evolução para expressões.

Melhorias e evolução - Testes - PI II: Validação final	Garantir qualidade após as mudanças da PI II.	Reteste dos cenários críticos e dos fluxos principais.
Entrega PI II: Consolidação do aplicativo e documentação	Fechar versão para entrega acadêmica.	Build estável, README, documentação do projeto revisados e preparação da apresentação final da disciplina.

Assim, o cronograma cobre o ciclo completo desde a implementação até a entrega, incluindo duas rodadas de testes (pós-MVP e pós-evolução na PI II), a etapa de encerramento da Atividades de Extensão II com o cliente e, na sequência, a evolução técnica do aplicativo com vista à entrega acadêmica.

6.3 Implementação técnica

Plataforma: aplicativo Android nativo em Java 8 (compileOptions 1.8), minSdk 21, compileSdk / targetSdk 34, applicationId com.example.librasrecognition, build com Gradle e catálogo de versões (libs.*).

Arquitetura em camadas: MainActivity gerencia permissões de câmera, ciclo de vida CameraX (pré-visualização + análise), troca de câmera frontal/traseira, menu de ajuda (HelpActivity) e atualização da UI. O processamento de cada quadro corre num ExecutorService de thread única para não bloquear a UI. HandDetectionProcessor converte o ImageProxy (YUV) em Bitmap, aplica rotação/espelhamento e orquestra inferência YOLO (TFLite) e MediaPipe Hands. HandOverlayView desenha os landmarks. O modelo e os rótulos vêm de app/src/main/assets/ (best_float32.tflite, labels.txt).

Pipeline de dados (por frame):

- CameraX: ImageAnalysis com STRATEGY_KEEP_ONLY_LATEST (descarta frames antigos se o processamento atrasar).

- Conversão YUV_420_888 → NV21 → JPEG em memória → Bitmap (compressão JPEG qualidade 100).
- Ajuste de orientação: `Matrix.postRotate(rotationDegrees)` e `postScale(-1, 1)` (imagem espelhada no eixo horizontal).
- Classificação: `GestureClassifier.predictWithScores(bitmap)` — letterbox para 640×640, preenchimento cinzento RGB(114,114,114), normalização de canais NCHW em float [0,1], execução do TensorFlow Lite Interpreter com 4 threads.
- Pós-processamento YOLO: leitura do tensor de saída em formato [1, 4+nc, anchors] ou [1, anchors, 4+nc] (detecção automática em `parseOutputShape`), aplicação de sigmoid às pontuações de classe quando os valores brutos não estão já em [0,1], filtro por limiar de confiança 0,35, NMS com IoU 0,45, escolha da melhor detecção e mapeamento do `classId` para o rótulo em `labels.txt`.
- MediaPipe Hands: (`setStaticImageMode(false)`, máx. 1 mão, confianças mínimas de detecção e tracking 0,8, `setRunOnGpu(true)`), com resultados enviados à UI na thread principal via Handler.

Treino / dados: pasta `yolo-train/dataset/` com `data.yaml` (Roboflow: projeto “libras”, versão 2, licença CC BY 4.0), 25 nomes de classe alinhados ao `labels.txt`. O export para TFLite float32 embutido no APK é o artefacto usado em tempo de execução.

Condição de carregamento do modelo: o classificador só considera carregamento válido se o interpretador existir e o número de linhas em `labels.txt` coincidir com `numClasses` inferido do tensor de saída, caso contrário registra erro nos logs (“Esperado X labels; arquivo tem Y”).

6.4 Limitações das tecnologias e do desenho atual

- Âmbito semântico: o modelo classifica gestos fixos (rótulos do treino), não “Libras geral” nem frases espontâneas fora das classes, parte dos rótulos são palavras/expressões treinadas, não só letras isoladas.

- Android apenas: não há implementação iOS no repositório, minSdk 21 abrange aparelhos muito antigos com CPU/GPU fracos — a latência e a taxa de frames analisáveis variam muito.
- Inferência no dispositivo: o código usa CPU multi-thread (`setNumThreads(4)`), sem delegados explícitos (GPU/NNAPI) no Interpreter — desempenho depende do chip e da carga térmica.
- CameraX + análise: `KEEP_ONLY_LATEST` melhora a responsividade mas perde frames, pode haver saltos na suavidade do rastreo.
- Conversão YUV → JPEG → Bitmap: é pesada e com perdas, aumenta latência e uso de memória em relação a um caminho YUV/RGB direto.
- MediaPipe: limiares 0,8 para detecção e tracking exigem mão bem visível, uma só mão, em dispositivos sem GPU adequada pode haver falhas.
- Desacoplamento YOLO vs landmarks: o rótulo/gesto vem do quadro completo (YOLO), o overlay vem do MediaPipe. Se o MediaPipe perder a mão mas o YOLO ainda “vir” um gesto (ou o contrário), a experiência pode ficar incoerente (texto vs ausência de landmarks).
- Alinhamento modelo/rótulos: qualquer retreino que altere nc ou a ordem das classes obriga a atualizar `labels.txt` em sincronia com o `.tflite`, senão `loaded` fica falso.

6.5 Erros e obstáculos

Área	Erro / Obstáculo
Ambiente de build	Android Gradle Plugin 8.x com JDK 17, falhas de sync resolvidas com JDK correto e Invalidate Caches.
Dispositivo	<code>INSTALL_FAILED_*</code> , drivers USB no Windows, adb devices, desinstalar versão antiga com o mesmo <code>applicationId</code> .
Câmera	Permissões negadas, câmera ocupada por outra app, erros em <code>bindToLifecycle</code> (tratados com log e Toast em <code>MainActivity</code>).
Integração TFLite + YOLO	Ajuste do layout do tensor de saída

	(channels-first vs channels-last), escala de caixas (normalizadas vs pixels — o código trata coordenadas $\leq 1,5$ como normalizadas), necessidade de NMS e limiares estáveis.
Qualidade do modelo	Confusões entre classes parecidas, sensibilidade a iluminação, fundo, pele, distância, dataset Roboflow com viés de contexto de captura.
Qualidade de código / depuração	No HandDetectionProcessor, o listener do MediaPipe registra “Nenhuma mão detectada” mesmo quando há resultado — mensagem de log enganadora para depuração (vale mencionar como “dívida técnica / ruído em logs”).